

Validación de modelos usando etiquetas de entrenamiento mutadas

Introducción

Los **modelos de validación con conjuntos de test** (out-of-sample validation) tienen varias limitaciones:

- El conjunto de test puede ser demasiado pequeño para representar la distribución real de los datos.
- La precisión puede tener una gran varianza entre diferentes ejecuciones.
- Los patrones se seleccionan aleatoriamente, por lo que, puede tener un sesgo.
- Al usar un conjunto fijo de test, puede provocar sobreentrenamiento incluso si los datos de test no son usados en el entrenamiento.

El **método de validación por mutación** se basa en dos métodos de ingeniería del software: *Mutation Testing* y *Metamorphic Testing*.

Mutation Testing muta un programa haciendo cambios pequeños y reejecuta los tests para comprobar el comportamiento que ha tenido esos cambios.

Metamorphic Testing detecta los errores de un programa comprobando la relación que se produce entre la entrada y la salida si cambiamos la entrada. Es decir, si cambiamos la entrada, comprobamos cómo cambia la salida.

El **Mutation Validation** muta las etiquetas de los datos de entrenamiento y reentrena el modelo usando estos datos mutados, entonces mide el cambio en el desempeño del modelo.

La teoría es que si el modelo es bueno, entonces no debería cambiar mucho cuando cambiemos algunas etiquetas. Si cambia mucho significaría que se produce un sobreentrenamiento.

Las **técnicas de validación tradicionales** no siempre funcionan bien en modelos modernos como redes neuronales profundas. Esto es debido a que podrían memorizar ciertas etiquetas.

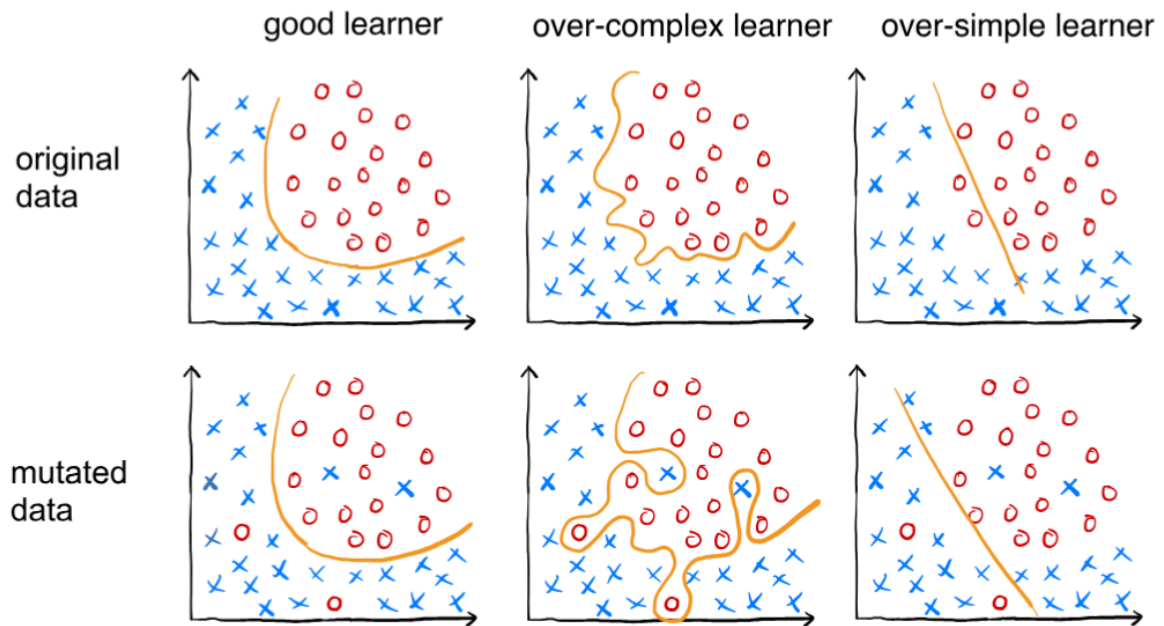
El método de validación MV es mejor por 3 razones:

- Un buen modelo debe ajustarse a la distribución real de los datos, no a los patrones en específico. MV detecta si está aprendiendo demasiado de los patrones.
- MV detecta mejor cuándo un modelo es demasiado complejo y empieza a sobreajustar.

- MV es estable (no cambia mucho con pequeñas variaciones).

Mutation Validation

Intuición general



Hay 3 posibilidades para un modelo entrenado:

- **Buen modelo:** Si validamos, tanto el modelo original como el modelo entrenado con las etiquetas mutadas, con los datos de entrenamiento originales, ambos deberían tener un buen rendimiento, ya que, el modelo no debería ajustarse a ciertas etiquetas mutadas aleatoriamente.
- **Modelo sobreajustado:** En este caso, el modelo original tendría un buen rendimiento, pero el modelo entrenado con las etiquetas mutadas y validado con los originales perdería bastante precisión.
- **Modelo poco entrenado:** En este caso, ambos modelos tendrían un mal rendimiento.

Mutation Validation

MV combina las técnicas de *mutation testing* y *metamorphic testing*.

La técnica de **mutation testing** crea mutantes inyectando fallos en un programa y reejecutando dicho programa para comprobar si los test detectan esos fallos.

La técnica de **metamorphic realtion** especifica como un cambio en la entrada debería resulta en un cambio en la salida.

En MV tratamos un modelo como un programa para testear, los datos de entrenamiento como la entrada y el comportamiento del programa como la salida.

Los cambios en la entrada se realizan mutando datos de entrenamiento, cada dato mutado se denomina un **mutante**. Los cambios en la salida se definen como el cambio en el rendimiento de los modelos entrenados.

Sean:

- S el dataset original.
- S_n el dataset mutado.
- n el grado de mutación. Porcentaje de datos a mutar.
- $f(S)$ el modelo entrenado con el dataset original.
- $f(S_n)$ el modelo entrenado con el dataset mutado.
- $T_S(f(S))$ la precisión del modelo $f(S)$ sobre el dataset original. Métrica clásica de precisión de un modelo.
- $T_S(f(S_n))$ la precisión del modelo $f(S_n)$ sobre el dataset original. Debería seguir siendo alta si el modelo es bueno, ya que no se adapta a los datos mutados.
- $T_{S_n}(f(S_n))$ la precisión del modelo $f(S_n)$ sobre el dataset mutado. Si es muy alto, el modelo ha memorizado el ruido.

La **nueva medida** usada para evaluar el modelo con esta nueva técnica es la siguiente:

$$m = (1 - 2n)T_S(f(S_n)) + T_{S(f(S))} - T_{S_n}(f(S_n)) + n$$

- Si m es cercano a 0, el modelo es malo, tiene sobreajuste o es débil.
- Si m es cercano a 1, el modelo es robusto y bueno.

Experimentos del paper

En el paper, para los experimentos hacen lo siguiente:

- n lo ponen como 0.2 para indicar que se van a mutar el 20% de los datos.
- Para la mutación se hace lo siguiente:
 1. Se hace una lista de las posibles etiquetas.
 2. Por cada dato a mutar se cambia su etiqueta por la siguiente de la lista (y la última con la primera).